Lab 2: Debugging

Date: 09/10/2026
Environment: macOS (Apple Silicon, arm64), Python 3.10.13 with pdb, lldb, Apple clang.

Task 1: Sorting Algorithms (debugged with pdb)

I translated the pseudocode into Python exactly as given (sorts_buggy.py) and ran it. fast_sort([3,1,2]) crashed with “RecursionError: maximum recursion depth exceeded”, and slow_sort([3,4,1,2]) returned [1, 1, 3, 4], which is wrong.

[Fig 1: Python version and the buggy code with line numbers]
[Fig 2: The failure, RecursionError and the output [1, 1, 3, 4]]

I then used pdb (python3 -m pdb driver1.py and driver2.py) to set breakpoints and inspect the variables. This found four bugs.

```
mkdir -p ~/lab2/task1 && cd ~/lab2/task1
Python 3.10.13
[mananchavda@Manans-MacBook-Air-1051 task1 % >....
[def slow_sort(arr):
    if len(arr) <= 1:
        return arr
[    mid = len(arr) // 2
    left = slow_sort(arr[:mid])
    right = slow_sort(arr[mid:])
    return add_and_rule(left, right)

def add_and_rule(left, right):
    result = []
    i = 0
    j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[i])
            j += 1
    result += left[i:]
    result += right[j:]
    return result
EOF
mananchavda@Manans-MacBook-Air-1051 task1 % cat -n sorts_buggy.py | sed -n '1,15p;25,38p'
     1  def fast_sort(arr, low, high):
     2      if low < high:
     3          p = divide_and_rule(arr, low, high)
     4          fast_sort(arr, low, p)
[    5          fast_sort(arr, p + 1, high)
[    6
     7  def divide_and_rule(arr, low, high):
     8      p = arr[high]
     9      i = low - 1
    10      for j in range(low, high + 1):
    11          if arr[j] <= p:
    12              i += 1
    13              arr[i], arr[j] = arr[j], arr[i]
    14      arr[i], arr[high] = arr[high], arr[i]
    15      return i
    25  def add_and_rule(left, right):
    26      result = []
    27      i = 0
    28      j = 0
    29      while i < len(left) and j < len(right):
    30          if left[i] <= right[j]:
    31              result.append(left[i])
    32              i += 1
    33          else:
    34              result.append(right[i])
    35              j += 1
    36      result += left[i:]
    37      result += right[j:]
    38      return result
mananchavda@Manans-MacBook-Air-1051 task1 % cat > driver1.py << 'EOF'
from sorts_buggy import fast_sort
a = [3, 1, 2]
fast_sort(a, 0, 2)
print("fast_sort result:", a)
EOF

cat > driver2.py << 'EOF'
```

Bug 1: divide_and_rule, the loop goes one step too far (line 10)

I set breakpoints at lines 11 and 14. On the third hit, “p j, i, arr” showed (2, 0, [1, 3, 2]). Here j = 2 equals high, so the pivot itself was being compared and swapped inside the loop.

Fix: change the loop to range(low, high).

```
EOF
mananchavda@Manans-MacBook-Air-1051 task1 % python3 driver1.py 2>&1 | tail -5
python3 driver2.py
  File "/Users/mananchavda/lab2/task1/sorts_buggy.py", line 3, in fast_sort
    p = divide_and_rule(arr, low, high)
  File "/Users/mananchavda/lab2/task1/sorts_buggy.py", line 10, in divide_and_rule
    for j in range(low, high + 1):
RecursionError: maximum recursion depth exceeded in comparison
slow_sort result: [1, 1, 3, 4]
mananchavda@Manans-MacBook-Air-1051 task1 % python3 -m pdb driver1.py
> /Users/mananchavda/lab2/task1/driver1.py(1)<module>()
-> from sorts_buggy import fast_sort
(Pdb) b sorts_buggy.py:11
Breakpoint 1 at /Users/mananchavda/lab2/task1/sorts_buggy.py:11
(Pdb) b sorts_buggy.py:14
Breakpoint 2 at /Users/mananchavda/lab2/task1/sorts_buggy.py:14
(Pdb) c
> /Users/mananchavda/lab2/task1/sorts_buggy.py(11)divide_and_rule()
-> if arr[j] <= p:
(Pdb) p j, i, arr
(0, -1, [3, 1, 2])
(Pdb) c
> /Users/mananchavda/lab2/task1/sorts_buggy.py(11)divide_and_rule()
-> if arr[j] <= p:
(Pdb) p j, i, arr
(1, -1, [3, 1, 2])
(Pdb) c
> /Users/mananchavda/lab2/task1/sorts_buggy.py(11)divide_and_rule()
-> if arr[j] <= p:
(Pdb) p j, i, arr
(2, 0, [1, 3, 2])
(Pdb) c
> /Users/mananchavda/lab2/task1/sorts_buggy.py(14)divide_and_rule()
-> arr[i], arr[high] = arr[high], arr[i]
(Pdb) p i, arr
(1, [1, 2, 3])
(Pdb) n
> /Users/mananchavda/lab2/task1/sorts_buggy.py(15)divide_and_rule()
-> return i
(Pdb) p arr
[1, 3, 2]
(Pdb) q
mananchavda@Manans-MacBook-Air-1051 task1 %
```

Bug 2: divide_and_rule, wrong pivot placement and return value (lines 14 and 15)

At line 14, “p i, arr” showed (1, [1, 2, 3]), which is already correctly partitioned with the pivot 2 at index 1. After pressing n, “p arr” showed [1, 3, 2]. The swap had moved the pivot back to the end and broken the order. The code swapped arr[i] and returned i, but the pivot belongs at i + 1.

Fix: swap arr[i+1] with arr[high] and return i + 1.

```
mananchavda@Manans-MacBook-Air-1051 task1 % python3 -m pdb driver1.py
> /Users/mananchavda/lab2/task1/driver1.py(1)<module>()
-> from sorts_buggy import fast_sort
(Pdb) b sorts_buggy.py:11
Breakpoint 1 at /Users/mananchavda/lab2/task1/sorts_buggy.py:11
(Pdb) b sorts_buggy.py:14
Breakpoint 2 at /Users/mananchavda/lab2/task1/sorts_buggy.py:14
(Pdb) c
> /Users/mananchavda/lab2/task1/sorts_buggy.py(11)divide_and_rule()
-> if arr[j] <= p:
(Pdb) p j, i, arr
(0, -1, [3, 1, 2])
(Pdb) c
> /Users/mananchavda/lab2/task1/sorts_buggy.py(11)divide_and_rule()
-> if arr[j] <= p:
(Pdb) p j, i, arr
(1, -1, [3, 1, 2])
(Pdb) c
> /Users/mananchavda/lab2/task1/sorts_buggy.py(11)divide_and_rule()
-> if arr[j] <= p:
(Pdb) p j, i, arr
(2, 0, [1, 3, 2])
(Pdb) c
> /Users/mananchavda/lab2/task1/sorts_buggy.py(14)divide_and_rule()
-> arr[i], arr[high] = arr[high], arr[i]
(Pdb) p i, arr
(1, [1, 2, 3])
(Pdb) n
> /Users/mananchavda/lab2/task1/sorts_buggy.py(15)divide_and_rule()
-> return i
(Pdb) p arr
[1, 3, 2]
(Pdb) q
mananchavda@Manans-MacBook-Air-1051 task1 % python3 -m pdb driver1.py
> /Users/mananchavda/lab2/task1/driver1.py(1)<module>()
-> from sorts_buggy import fast_sort
(Pdb) b sorts_buggy.py:4
Breakpoint 1 at /Users/mananchavda/lab2/task1/sorts_buggy.py:4
(Pdb) c
> /Users/mananchavda/lab2/task1/sorts_buggy.py(4)fast_sort()
-> fast_sort(arr, low, p)
(Pdb) pp low, p, high
(0, 1, 2)
(Pdb) pp low, p, high
(0, 1, 2)
(Pdb) c
> /Users/mananchavda/lab2/task1/sorts_buggy.py(4)fast_sort()
-> fast_sort(arr, low, p)
(Pdb) pp low, p, high
(0, 1, 1)
(Pdb) c
> /Users/mananchavda/lab2/task1/sorts_buggy.py(4)fast_sort()
-> fast_sort(arr, low, p)
(Pdb) pp low, p, high
(0, 1, 1)
(Pdb) c
> /Users/mananchavda/lab2/task1/sorts_buggy.py(4)fast_sort()
-> fast_sort(arr, low, p)
(Pdb) pp low, p, high
(0, 1, 1)
(Pdb) q
mananchavda@Manans-MacBook-Air-1051 task1 %
```

Bug 3: fast_sort, infinite recursion (line 4)

I set a breakpoint at line 4. “pp low, p, high” printed (0, 1, 2), then (0, 1, 1) again and again. The range never shrinks, because fast_sort(arr, low, p) includes the pivot, which is already in its final place.

Fix: call fast_sort(arr, low, p - 1).

[Fig 4: Bug 3, with (0, 1, 1) repeating]

```
(Pdb) q
mananchavda@Manans-MacBook-Air-1051 task1 % python3 -m pdb driver2.py
> /Users/mananchavda/lab2/task1/driver2.py(1)<module>()
-> from sorts_buggy import slow_sort
(Pdb) b sorts_buggy.py:34
Breakpoint 1 at /Users/mananchavda/lab2/task1/sorts_buggy.py:34
(Pdb) c
> /Users/mananchavda/lab2/task1/sorts_buggy.py(34)add_and_rule()
-> result.append(right[i])
(Pdb) pp left, right, i, j, result
([3, 4], [1, 2], 0, 0, [])
(Pdb) p right[i], right[j]
(1, 1)
(Pdb) c
> /Users/mananchavda/lab2/task1/sorts_buggy.py(34)add_and_rule()
-> result.append(right[i])
(Pdb) pp left, right, i, j, result
([3, 4], [1, 2], 0, 1, [1])
(Pdb) p right[i], right[j]
(1, 2)
(Pdb) q
```

Bug 4: add_and_rule, wrong index (line 34)

At the second hit of the breakpoint at line 34, “pp left, right, i, j, result” showed ([3, 4], [1, 2], 0, 1, [1]), and “p right[i], right[j]” showed (1, 2). The code appended right[i] (1 again) instead of right[j] (2). This produced the duplicate in [1, 1, 3, 4].

Fix: use result.append(right[j]).

[Fig 5: Bug 4]

Summary of bugs and fixes

1. divide_and_rule: the loop includes the pivot (high + 1). Fix: range(low, high).
2. divide_and_rule: the pivot is swapped at i and i is returned. Fix: swap arr[i+1] and return i+1.
3. fast_sort: the recursion on (low, p) never shrinks. Fix: use (low, p - 1).
4. add_and_rule: right[i] is used instead of right[j]. Fix: use right[j].

Result

After the four fixes (sorts.py), fast_sort([3,1,2]) gives [1, 2, 3] and slow_sort([3,4,1,2]) gives [1, 2, 3, 4]. 1000 random tests against Python’s sorted() all pass.

[Fig 6: Fixed code and “All 1000 random tests passed”]

[Fig 7: Before and after drivers, and cat -n sorts.py]

```
(Pdb) q
mananchavda@Manans-MacBook-Air-1051 task1 % cd ~/lab2/task1
cat > sorts.py << 'EOF'
def fast_sort(arr, low, high):
    if low < high:
        p = divide_and_rule(arr, low, high)
        fast_sort(arr, low, p - 1)          # fix 3: pivot already in place
        fast_sort(arr, p + 1, high)

def divide_and_rule(arr, low, high):
    p = arr[high]
    i = low - 1
    for j in range(low, high):              # fix 1: stop before the pivot
        if arr[j] <= p:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i + 1], arr[high] = arr[high], arr[i + 1]   # fix 2: pivot goes to i+1
    return i + 1                                     # fix 2

def slow_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = slow_sort(arr[:mid])
    right = slow_sort(arr[mid:])
    return add_and_rule(left, right)

def add_and_rule(left, right):
    result = []
    i = 0
    j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])         # fix 4: use j, not i
            j += 1
    result += left[i:]
    result += right[j:]
    return result

if __name__ == "__main__":
    import random
    a = [3, 1, 2]
    fast_sort(a, 0, 2)
    print("fast_sort [3,1,2] ->", a)
    print("slow_sort [3,4,1,2] ->", slow_sort([3, 4, 1, 2]))
    for _ in range(1000):
        x = [random.randint(-50, 50) for _ in range(random.randint(0, 20))]
        y = x[:]
        fast_sort(y, 0, len(y) - 1)
        assert y == sorted(x), ("fast_sort failed", x, y)
        assert slow_sort(x) == sorted(x), ("slow_sort failed", x)
    print("All 1000 random tests passed")
EOF
mananchavda@Manans-MacBook-Air-1051 task1 % python3 sorts.py
fast_sort [3,1,2] -> [1, 2, 3]
slow_sort [3,4,1,2] -> [1, 2, 3, 4]
All 1000 random tests passed
mananchavda@Manans-MacBook-Air-1051 task1 %
```

Task 2: Memory Corruption (debugged with lldb)

The task was changed to normal debugging, so I used lldb. (Before that I tried rr and WarpSpeed. WarpSpeed built on my Mac but failed at runtime with “no room in the host address space for the guest’s malloc heap.”)

The program with line numbers:

[Fig 8: corruption.c shown with cat -n]

Step 1: Compile and run

I compiled with gcc -g -O0 corruption.c -o corruption and ran it. Student 1’s id changed from 1002 to 1007. The program printed “ERROR: Student 1’s ID was corrupted! Expected 1002, got 1007” and exited with code 1.

[Fig 9: Corrupted run (id=1007, exit code 1) and the start of the lldb session]

```
(lldb) target create "./corruption"
error: 'corruption' does not exist
(lldb) quit
mananchavda@Manans-MacBook-Air-1051 task1 % mkdir -p ~/lab2/task2 && cd ~/lab2/task2
ls
mananchavda@Manans-MacBook-Air-1051 task2 % cat > corruption.c << 'EOF'
#include <stdio.h>

typedef struct {
    int id;
    int scores[3];
} Student;

Student students[2];

void init() {
    students[0].id = 1001;
    students[0].scores[0] = 85;
    students[0].scores[1] = 92;
    students[0].scores[2] = 78;

    students[1].id = 1002;
    students[1].scores[0] = 90;
    students[1].scores[1] = 88;
    students[1].scores[2] = 95;
}

void curve_scores(int student_idx, int curve) {
    for (int i = 0; i < 4; i++) {
        students[student_idx].scores[i] += curve;
    }
}

int main() {
    init();
    printf("=== Initial state ===\n");
    printf("Student 0: id=%d\n", students[0].id);
    printf("Student 1: id=%d\n", students[1].id);

    curve_scores(0, 5);

    printf("\n=== After curving ===\n");
    printf("Student 0: id=%d\n", students[0].id);
    printf("Student 1: id=%d\n", students[1].id);

    if (students[1].id != 1002) {
        printf("\nERROR: Student 1's ID was corrupted! Expected 1002, got %d\n",
               students[1].id);
        return 1;
    }
    return 0;
}
EOF
mananchavda@Manans-MacBook-Air-1051 task2 % gcc -g -O0 corruption.c -o corruption
./corruption
echo "exit code: $?"
=== Initial state ===
Student 0: id=1001
Student 1: id=1002
```

Steps 2 and 3: Set the watchpoint

In lldb I ran “b main” and “run”, then “watchpoint set variable students[1].id”. The watchpoint was created at address 0x100008010.

First stop: in init(), old value 0, new value 1002. This is the normal initialization, so it is not a bug.

[Fig 10: Watchpoint created at 0x100008010, and the first hit (0 to 1002) in init()]

```
old value: 0
new value: 1002
Process 24743 stopped
* thread #1, queue = 'com.apple.main-thread', stop reason = watchpoint 1
    frame #0: 0x0000000100000528 corruption`init at corruption.c:17:27
   14       students[0].scores[2] = 78;
   15
   16       students[1].id = 1002;
-> 17       students[1].scores[0] = 90;
   18       students[1].scores[1] = 88;
   19       students[1].scores[2] = 95;
   20   }
Target 0: (corruption) stopped.
(lldb) continue
Process 24743 resuming
=== Initial state ===
Student 0: id=1001
Student 1: id=1002

Watchpoint 1 hit:
old value: 1002
new value: 1007
Process 24743 stopped
* thread #1, queue = 'com.apple.main-thread', stop reason = watchpoint 1
    frame #0: 0x0000000100000590 corruption`curve_scores(student_idx=0, curve=5) at corruption.c:25:5
   22   void curve_scores(int student_idx, int curve) {
   23       for (int i = 0; i < 4; i++) {
   24           students[student_idx].scores[i] += curve;
-> 25       }
   26   }
   27
   28   int main() {
Target 0: (corruption) stopped.
(lldb) bt
* thread #1, queue = 'com.apple.main-thread', stop reason = watchpoint 1
  * frame #0: 0x0000000100000590 corruption`curve_scores(student_idx=0, curve=5) at corruption.c:25:5
    frame #1: 0x0000000100000628 corruption`main at corruption.c:34:5
    frame #2: 0x000000018114fe00 dyld`start + 6992
(lldb) frame variable
(int) student_idx = 0
(int) curve = 5
(int) i = 3
(lldb) p &students[0].scores[3]
(int *) 0x0000000100008010
(lldb) p &students[1].id
(int *) 0x0000000100008010
(lldb) quit
Quitting LLDB will kill one or more processes. Do you really want to proceed: [Y/n] Y
mananchavda@Manans-MacBook-Air-1051 task2 % cd ~/lab2/task2
sed 's/i < 4/i < 3/' corruption.c > corruption_fixed.c
gcc -g -O0 corruption_fixed.c -o corruption_fixed
./corruption_fixed
echo "exit code: $?"
=== Initial state ===
Student 0: id=1001
Student 1: id=1002

=== After curving ===
Student 0: id=1001
Student 1: id=1002
exit code: 0
mananchavda@Manans-MacBook-Air-1051 task2 %
```

Steps 4 and 5: Find the corruption

Second stop: old value 1002, new value 1007, inside curve_scores(student_idx=0, curve=5) at corruption.c:25. This is the corruption. It happens in a function that is only supposed to touch student 0.

"bt" shows curve_scores called from main. "frame variable" shows i = 3. Both "p &students[0].scores[3]" and "p &students[1].id" printed 0x0000000100008010.

[Fig 11: Second hit (1002 to 1007), bt, i = 3, identical addresses, and the fixed run with exit code 0]

Root cause

The scores array has 3 elements (indices 0 to 2), but the loop runs while i < 4. On the last iteration i = 3, so the code writes to scores[3], which is one element past the end of students[0]. In memory, that location is exactly students[1].id. The += 5 therefore changed 1002 into 1007. The two identical addresses prove this.

```
(lldb) target create "./corruption"
error: 'corruption' does not exist
(lldb) quit
mananchavda@Manans-MacBook-Air-1051 task1 % mkdir -p ~/lab2/task2 && cd ~/lab2/task2
ls
mananchavda@Manans-MacBook-Air-1051 task2 % cat > corruption.c << 'EOF'
#include <stdio.h>

typedef struct {
    int id;
    int scores[3];
} Student;

Student students[2];

void init() {
    students[0].id = 1001;
    students[0].scores[0] = 85;
    students[0].scores[1] = 92;
    students[0].scores[2] = 78;

    students[1].id = 1002;
    students[1].scores[0] = 90;
    students[1].scores[1] = 88;
    students[1].scores[2] = 95;
}

void curve_scores(int student_idx, int curve) {
    for (int i = 0; i < 4; i++) {
        students[student_idx].scores[i] += curve;
    }
}

int main() {
    init();
    printf("=== Initial state ===\n");
    printf("Student 0: id=%d\n", students[0].id);
    printf("Student 1: id=%d\n", students[1].id);

    curve_scores(0, 5);

    printf("\n=== After curving ===\n");
    printf("Student 0: id=%d\n", students[0].id);
    printf("Student 1: id=%d\n", students[1].id);

    if (students[1].id != 1002) {
        printf("\nERROR: Student 1's ID was corrupted! Expected 1002, got %d\n",
               students[1].id);
        return 1;
    }
    return 0;
}
EOF
mananchavda@Manans-MacBook-Air-1051 task2 % gcc -g -O0 corruption.c -o corruption
./corruption
echo "exit code: $?"
=== Initial state ===
Student 0: id=1001
Student 1: id=1002
```

Fix

Change i < 4 to i < 3. After recompiling (corruption_fixed.c), Student 1's id stays 1002 and the exit code is 0.

Note: compiling with -fsanitize=address would also report this as a global-buffer-overflow.

Conclusion

Using pdb, I found and fixed four separate bugs in the sorting code. It now matches Python's

sorted() on 1000 random tests. Using an lldb watchpoint, I traced the corruption of students[1].id to an out-of-bounds write in curve_scores, and fixed it by correcting the loop bound.